

Trabajo Práctico Integrador

Cierre de Sistemas Expertos: Agente Racional para el Mundo del Wumpus

Este trabajo cierra la unidad de Sistemas Expertos aplicando sus conceptos centrales —base de hechos, reglas de producción y motor de inferencia— a un agente que percibe un entorno parcialmente observable y decide sus acciones únicamente a partir de lo que puede deducir lógicamente.

1. Objetivo

Implementar, en grupos de hasta 4 integrantes, un agente racional que resuelva el problema del Mundo del Wumpus mediante los mismos mecanismos usados en un sistema experto: una base de conocimiento que almacena hechos (percepciones), un conjunto de reglas de producción, y un motor de inferencia que las aplica para deducir qué casillas son seguras.

Importante: no es un juego para jugar. El tablero se genera aleatoriamente y el agente actúa solo, guiado exclusivamente por inferencia lógica. El grupo observa y documenta cómo razona, no lo controla.

2. El entorno: Mundo de Wumpus

Repaso de las reglas del mundo (Russell & Norvig, cap. 7):

- Grilla de 4×4 casillas. El agente arranca en (1,1), mirando hacia el este.
- Un wumpus, ubicado aleatoriamente (nunca en (1,1)).
- Entre 1 y 3 pozos (pits), ubicados aleatoriamente (nunca en (1,1)).
- Una pieza de oro, ubicada aleatoriamente.
- El agente tiene una flecha para disparar en línea recta y matar al wumpus.

Percepciones (lo único que el agente puede sentir en su casilla actual)

Percepción	Significado	Aparece cuando...
Stench	Hedor	El wumpus está en una casilla adyacente (no diagonal)
Breeze	Brisa	Hay un pozo en una casilla adyacente
Glitter	Brillo	El oro está en la casilla actual
Bump	Golpe	El agente choca contra un límite del tablero
Scream	Grito	El wumpus murió por la flecha

Acciones disponibles

- Moverse hacia adelante
- Girar 90° a izquierda / derecha
- Disparar la flecha (una sola vez, en la dirección actual)
- Agarrar el oro (si está en la casilla actual)
- Salir de la cueva (solo desde (1,1))

3. Conexión con Sistemas Expertos

Antes de programar, es importante que el grupo identifique la equivalencia conceptual con lo visto en la unidad anterior:

Sistema experto clásico	Agente del Wumpus
Base de hechos	Percepciones acumuladas por cada casilla visitada
Reglas de producción (SI... ENTONCES...)	"SI hay Breeze en (x,y) ENTONCES al menos una casilla adyacente tiene pozo"
Motor de inferencia (encadenamiento hacia adelante)	Recorre las reglas y actualiza qué casillas son seguras, peligrosas o desconocidas
Conclusión / recomendación del sistema	Próxima acción del agente, elegida entre las casillas seguras

Esta tabla debe aparecer también en el informe final, con al menos un ejemplo propio del grupo (no copiado de esta guía).

4. Consigna de trabajo

Trabajo grupal, de hasta 4 integrantes. Se desarrolla en dos etapas:

Etapas 1 — Simulación por consola (obligatoria)

El tablero se genera aleatoriamente en cada corrida (posiciones válidas, nunca coincidiendo con (1,1)). El agente recorre el entorno de forma completamente autónoma, y el programa imprime en consola, paso a paso, la traza del razonamiento: percepción recibida, regla disparada, conclusión nueva, acción tomada.

Etapas 2 — Visualización (a elección del grupo)

Una vez que la Etapa 1 funciona correctamente, el grupo puede optar por:

- Representación ASCII del tablero en cada paso (una matriz de caracteres impresa en consola), o
- Interfaz gráfica simple (por ejemplo con Pygame, Tkinter, o similar).

Ambas opciones son válidas y se evalúan igual siempre que no reemplacen el trabajo de la Etapa 1, que es la que realmente importa para esta unidad.

5. Arquitectura del proyecto

Se pide dividir el trabajo en módulos, no un único archivo. Cada módulo corresponde a un concepto de sistemas expertos, lo que además permite repartir el trabajo dentro del grupo y probar cada parte por separado.

```
wumpus_agente/  
├── main.py           # Orquesta la simulación, imprime la traza  
├── tablero.py        # El mundo real: wumpus, pozos, oro, percepciones  
├── agente.py         # Estado físico del agente (posición, flecha, oro)  
├── base_conocimiento.py # Hechos y conclusiones ("base de hechos")  
├── reglas.py         # Reglas de producción SI-ENTONCES  
├── motor_inferencia.py # Motor de inferencia (forward chaining)  
└── visualizacion.py  # ASCII o gráfica (Etapa 2, opcional)
```

Regla de oro: ningún módulo debe conocer detalles internos de otro más allá de lo estrictamente necesario. Por ejemplo, el agente nunca debe acceder directamente al tablero real (eso sería "hacer trampa" mirando dónde está el wumpus); solo puede llamar a percibir(x, y).

6. Detalle de cada módulo, con ejemplo de código

Los siguientes fragmentos son puntos de partida, no soluciones completas. Sirven para mostrar el estilo y la responsabilidad de cada archivo.

6.1. tablero.py — el mundo real

Genera el mundo aleatoriamente y expone únicamente lo que el agente tiene derecho a percibir. Guarda la verdad absoluta, pero no se la entrega al agente.

```
import random  
  
TAMANIO = 4  
  
def generar_tablero(cant_pozos=2):  
    casillas = [(x, y) for x in range(1, TAMANIO+1)  
                 for y in range(1, TAMANIO+1)  
                 if (x, y) != (1, 1)]  
    random.shuffle(casillas)  
    wumpus = casillas.pop()  
    pozos = [casillas.pop() for _ in range(cant_pozos)]  
    oro = casillas.pop()  
    return {'wumpus': wumpus, 'pozos': pozos, 'oro': oro}  
  
def adyacentes(x, y):  
    candidatas = [(x+1,y), (x-1,y), (x,y+1), (x,y-1)]  
    return [(a, b) for a, b in candidatas  
            if 1 <= a <= TAMANIO and 1 <= b <= TAMANIO]  
  
def percibir(tablero, x, y):
```

```
vecinas = adyacentes(x, y)
return {
    'breeze': any(v in tablero['pozos'] for v in vecinas),
    'stench': tablero['wumpus'] in vecinas,
    'glitter': (x, y) == tablero['oro'],
}
```

tablero.py — genera el mundo y responde "¿qué se percibe en (x,y)?" sin revelar nada más.

6.2. agente.py — estado físico del agente

Guarda posición y estado, y ejecuta acciones. No decide nada por sí mismo: solo hace lo que el motor de decisión (en main.py) le indique.

```
class Agente:
    def __init__(self):
        self.x, self.y = 1, 1
        self.tiene_oro = False
        self.tiene_flecha = True
        self.vivo = True

    def mover_a(self, x, y):
        self.x, self.y = x, y

    def agarrar_oro(self):
        self.tiene_oro = True
```

agente.py — estado y ejecución de acciones, sin lógica de decisión.

6.3. base_conocimiento.py — la base de hechos

Almacena, por cada casilla visitada, la percepción recibida, y las conclusiones que el motor de inferencia vaya deduciendo (segura / peligrosa / desconocida).

```
class BaseConocimiento:
    def __init__(self):
        self.percepciones = {}          # (x,y) -> {'breeze':..., 'stench':..., ...}
        self.seguras = {(1, 1)}
        self.peligrosas = set()
        self.visitadas = {(1, 1)}

    def registrar_percepcion(self, x, y, percepcion):
        self.percepciones[(x, y)] = percepcion
        self.visitadas.add((x, y))

    def marcar_segura(self, casilla):
        self.seguras.add(casilla)

    def marcar_peligrosa(self, casilla):
        self.peligrosas.add(casilla)

    def casillas_por_explorar(self):
        return self.seguras - self.visitadas - self.peligrosas
```

base_conocimiento.py — solo almacena y consulta hechos, no razona.

6.4. reglas.py — reglas de producción

Cada regla se escribe como una función independiente y legible, tal como conviene documentar en un sistema experto real. Reciben la casilla, sus adyacentes y la base de conocimiento, y devuelven qué conclusiones nuevas se pueden agregar.

```
from tablero import adyacentes

def regla_sin_breeze_ni_stench(casilla, base):
    """SI no hubo breeze ni stench en 'casilla'
       ENTONCES todas sus adyacentes son seguras."""
    x, y = casilla
    percepcion = base.percepciones[casilla]
    conclusiones = []
    if not percepcion['breeze'] and not percepcion['stench']:
        for vecina in adyacentes(x, y):
            conclusiones.append(('segura', vecina))
    return conclusiones

def regla_breeze(casilla, base):
    """SI hay breeze en 'casilla'
       ENTONCES al menos una adyacente no confirmada-segura es sospechosa de pozo."""
    x, y = casilla
    percepcion = base.percepciones[casilla]
    conclusiones = []
    if percepcion['breeze']:
        vecinas = adyacentes(x, y)
        sospechosas = [v for v in vecinas if v not in base.seguras]
        if len(sospechosas) == 1:
            conclusiones.append(('peligrosa', sospechosas[0]))
    return conclusiones

# reglas activas del sistema, en el orden en que se evalúan
REGLAS = [regla_sin_breeze_ni_stench, regla_breeze]
```

reglas.py — cada función es una regla SI-ENTONCES independiente y con docstring explicando su lógica.

A completar por el grupo: reglas equivalentes para stench (deducir dónde puede/no puede estar el wumpus) y para descartar sospechas cuando nueva información las contradice.

6.5. motor_inferencia.py — encadenamiento hacia adelante

Este es el corazón del sistema experto: recorre la lista de reglas y las aplica sobre cada casilla visitada, incorporando las conclusiones nuevas a la base de conocimiento. Así se ve, paso a paso, cómo se aplicaría una sola regla:

```
from reglas import REGLAS

def aplicar_reglas(base):
    """Recorre todas las casillas visitadas y dispara cada regla.
       Devuelve True si se agregó conocimiento nuevo (para repetir
       el ciclo, tal como hace un motor de encadenamiento hacia adelante)."""
    hubo_novedades = False
```

```
for casilla in base.visitadas:
    for regla in REGLAS:
        conclusiones = regla(casilla, base)

        for tipo, destino in conclusiones:
            if tipo == 'segura' and destino not in base.seguras:
                base.marcar_segura(destino)
                hubo_novedades = True
            elif tipo == 'peligrosa' and destino not in base.peligrosas:
                base.marcar_peligrosa(destino)
                hubo_novedades = True

return hubo_novedades

def inferir(base):
    """Aplica las reglas repetidamente hasta que no surjan más
    conclusiones nuevas (punto fijo), igual que un motor
    de inferencia de un sistema experto."""
    while aplicar_reglas(base):
        pass
```

motor_inferencia.py — el ciclo "aplicar reglas hasta que no haya novedades" es exactamente el forward chaining visto en la unidad.

Por qué el ciclo while: una sola pasada puede no alcanzar. Por ejemplo, marcar una casilla como segura puede habilitar recién en la siguiente pasada que otra regla deduzca algo sobre sus vecinas. El motor se detiene solo cuando llega a un punto fijo (ninguna regla aporta nada nuevo), tal como el encadenamiento hacia adelante clásico.

6.6. main.py — orquestación y traza

Arma las piezas, ejecuta el ciclo percibir → inferir → decidir → actuar, e imprime la traza legible para el informe.

```
from tablero import generar_tablero, percibir, adyacentes
from agente import Agente
from base_conocimiento import BaseConocimiento
from motor_inferencia import inferir

def elegir_accion(agente, base):
    candidatas = base.casillas_por_explorar()
    if candidatas:
        return ('mover', next(iter(candidatas)))
    return ('detener', None)

def simular():
    tablero = generar_tablero()
    agente = Agente()
    base = BaseConocimiento()
    paso = 1

    while agente.vivo and not agente.tiene_oro:
        percepcion = percibir(tablero, agente.x, agente.y)
        base.registrar_percepcion(agente.x, agente.y, percepcion)
        print(f"Paso {paso}: Agente en {(agente.x, agente.y)}. Percibe: {percepcion}")
```

```
inferir(base)

if percepcion['glitter']:
    agente.agarrar_oro()
    print(' -> Encontro el oro. Simulacion terminada con exito.')
    break

accion, destino = elegir_accion(agente, base)
if accion == 'mover':
    print(f' -> Accion elegida: mover a {destino} (casilla segura)')
    agente.mover_a(*destino)
else:
    print(' -> No quedan casillas seguras por explorar. Fin de la simulacion.')
    break

paso += 1

if __name__ == '__main__':
    simular()
```

main.py — versión mínima. El grupo debe agregar el manejo de peligro (wumpus/pozo), disparo de flecha y salida con el oro.

7. Formato de traza esperado

La consola debe mostrar, en cada paso, algo equivalente a esto (no hace falta el formato exacto, pero sí que se entienda percepción → regla → conclusión → acción):

```
Paso 3: Agente en (2, 1). Percibe: {'breeze': True, 'stench': False, 'glitter': False}
Regla disparada: regla_breeze
Conclusion nueva: (2, 2) marcada como PELIGROSA (posible pozo)
Accion elegida: mover a (1, 2) [casilla segura]
```

Este log es, en definitiva, el mismo tipo de traza que produciría cualquier sistema experto al explicar su razonamiento — por eso se pide explícitamente en el informe.

8. Reparto sugerido dentro del grupo (4 integrantes)

1. tablero.py + agente.py — modelado del mundo y del estado del agente.
2. reglas.py — traducir el razonamiento lógico del Wumpus World a reglas de producción.
3. base_conocimiento.py + motor_inferencia.py — el "corazón" de sistema experto.
4. main.py + visualizacion.py — orquestación, política de decisión, traza y (opcional) visual.

Se recomienda que cada integrante pueda probar su módulo de forma aislada antes de integrar, y que el informe final tenga una sección corta por persona explicando su parte.

9. Entregables

- Código fuente completo, organizado en los módulos indicados (o una organización equivalente justificada).
- Informe breve (PDF o Word) que incluya: tabla de equivalencia con sistemas expertos completada por el grupo, lista de reglas implementadas con una línea de justificación cada una, y una traza completa de al menos una corrida de ejemplo.

- Aclarar en el informe si se optó por visualización ASCII, gráfica, o ninguna (Etapa 2 es opcional).

10. Criterios de evaluación

Criterio	Qué se observa
Correctitud de las reglas	Las reglas deducen correctamente casillas seguras/peligrosas según las percepciones
Separación de responsabilidades	El agente no accede al tablero real; cada módulo cumple solo su rol
Claridad del motor de inferencia	El ciclo de encadenamiento hacia adelante es legible y se entiende cuándo se detiene
Calidad de la traza	Se puede seguir el razonamiento paso a paso desde la consola o el informe
Comportamiento del agente	Evita pozos y wumpus usando solo inferencia, sin mirar el tablero completo ni fuerza bruta
Trabajo en grupo	Reparto de módulos reflejado en el informe y en el historial de trabajo del grupo